SSH Internals Part 1: Protocol Architecture and the Cryptographic Handshake

medium.com/@pragalva.sapkota/ssh-internals-part-1-protocol-architecture-and-the-cryptographic-handshake-6ba9e5f8e16b

Pragalva Sapkota January 31, 2026

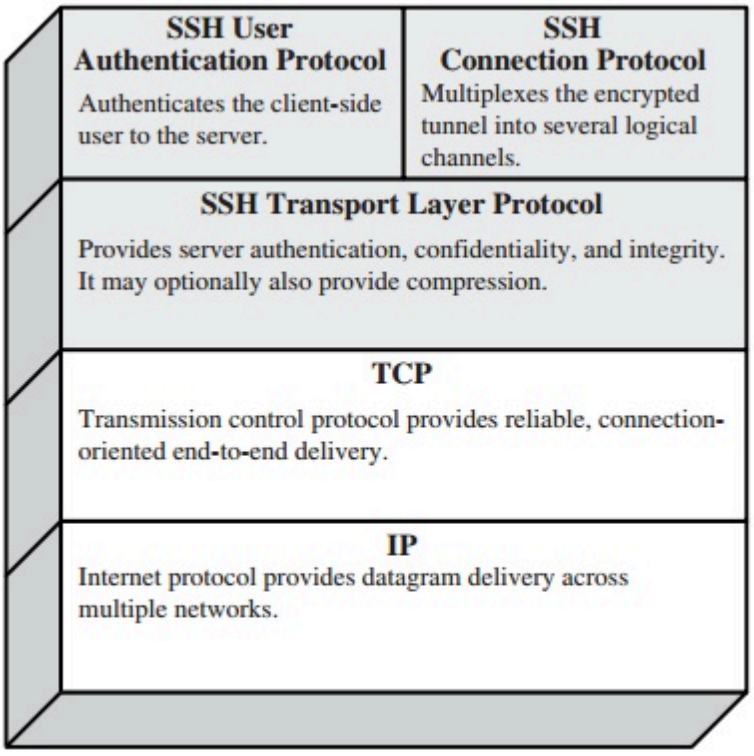


Figure 16.8 SSH Protocol Stack

Introduction

SSH is a protocol for secure remote login and other secure network services over an insecure network. I use SSH every day. If you are reading this, you probably do too. But how often do you look at your terminal and wonder: *how is this actually working?*

In this post, I am going to answer exactly that. We will dive into the **protocol architecture**, analyze the **packet structure**, and break down the **cryptographic handshake**. This is Part 1 of a series; Part 2 will cover the User Authentication Layer and Connection Multiplexing.

Table of Content:

The Beginning

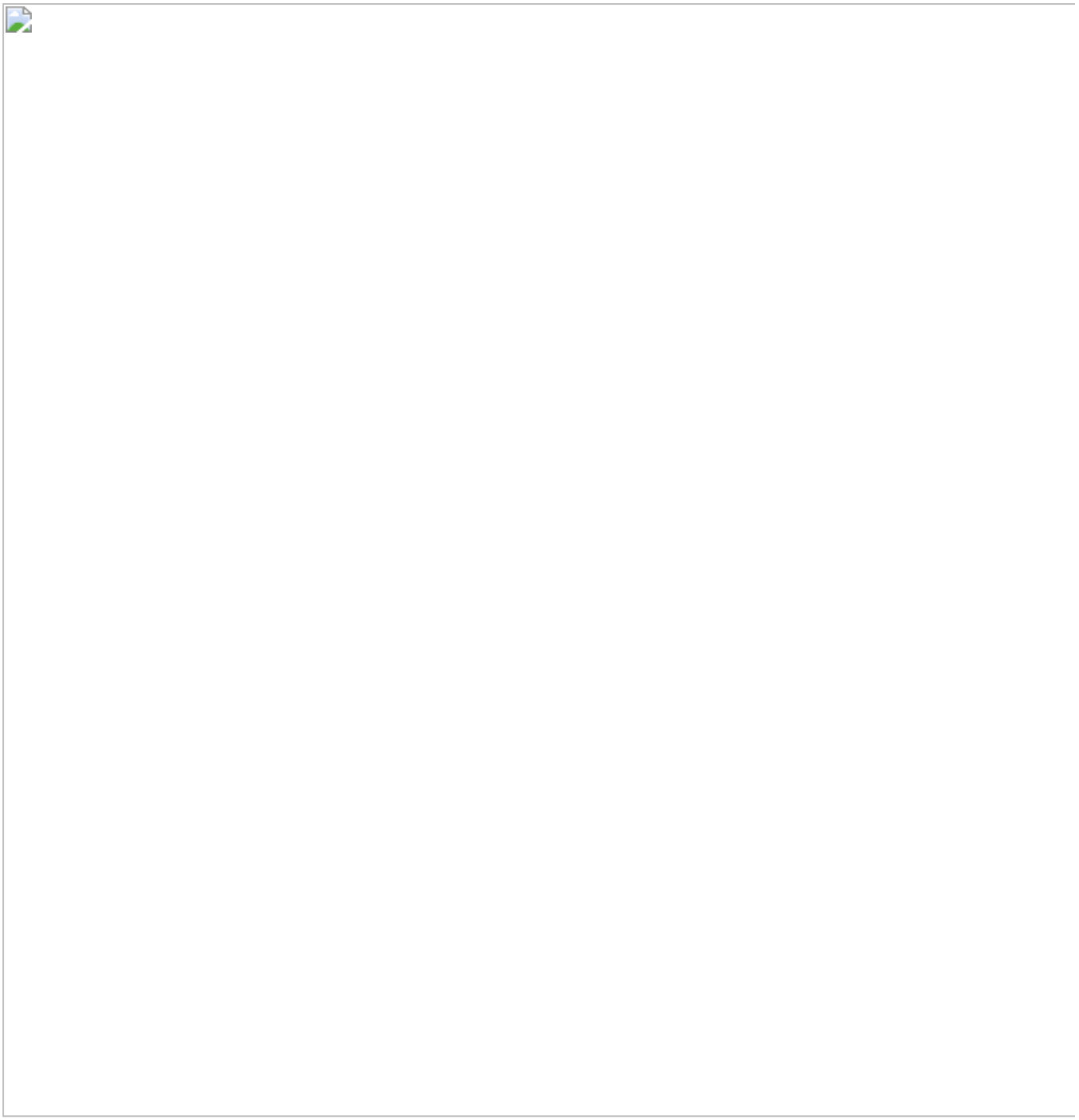
Before we dive straight into the architecture, let's take a moment to step back and look at where it all started.

Back in 1969, when the pioneers of the internet (then called ARPANET) were inventing ways for computers to talk to each other, they didn't want to sound like they were commanding people to follow their rules. So, they humbly named their documents **"Request for Comments"**.

The numbers (e.g., RFC 791) are assigned sequentially by the **IETF** (Internet Engineering Task Force). Once a new standard is proposed, it gets the next available number.

Because SSH is a massive protocol, it couldn't be summed up in a single document. Instead, the creators split it into four main RFCs that work together:

Press enter or click to view image in full size



RFC 4251 (Architecture of SSH Protocol)

As mentioned in the above table and the figure below, SSH consists of 3 major components that we will discuss furthermore.

1) The Transport Layer:

We can think of Transport Layer as a safe tunnel created between 2 buildings. The transport layer has 3 key responsibilities:
- **Confidentiality**: Encrypting data so no one can read it
- **Integrity**: Ensuring that the data hasn't been tampered
- **Server Authentication**: Preventing Man-in-the-Middle (MTM) attacks

The transport layer will typically run over a TCP/IP connection, but might also be used on top of any other reliable data stream. At this stage, the connection is encrypted, but the server has no idea who you (the user) are. It only knows that a secure channel exists. (more of it will be discussed in “The Cryptographic Handshake”)

2) The User Authentication Layer:

This layer runs inside the Transport Layer. It authenticates the client-side user to the server meaning it asks the client for proof of identity, this could be a password, a public key, or a host-based certificate.

3) The Connection Layer:

Once the Transport Layer has built the secure tunnel and the Auth Layer has let you in, the **Connection Layer** takes over. Its job is **multiplexing**: the ability to run multiple logical “channels” over that single encrypted connection.

Think of it like a fiber optic cable. The cable (the Transport Layer) is the physical link, but inside it, you can have hundreds of distinct strands of light (Channels) carrying different data simultaneously. Let’s look at a practical example:

Example: ssh -L 8080:localhost:5432 user@remote-server

This single command triggers the connection layer to manage multiple things at once. Here is the breakdown:

- Channel 0: SSH on default opens a terminal session so we can run commands on the server
- Channel 1: SSH starts lisetning to your computer’s port 8080.

Here the channel isn’t open yet, the moment we connect our local database to localhost:8080, the SSH client sends a request inside the tunnel for opening a tunnel on the server end. If the server accepts the request, we have 2 distinct streams of data (commands run in our terminal + the database queries) flowing over the same encrypted TCP connection.

Packet Structure of SSH

Up until now, we have discussed the architecture of the protocol the layers and the logic. But that is just the theory on paper.

To understand how SSH *actually* works on the wire, we need to leave the abstract diagrams behind and look at the raw implementation. We need to understand the **Binary Packet**.

Press enter or click to view image in full size

SSH uses a strict formatting for it’s packet and it can be categorized into:

1. Tells receiver how big the total packet it. Previously the length was not encrypted but now it's encrypted with some modern models like *chacha20-poly1305* to hide traffic patterns.
2. Tells how much "junk data" is present at the end of the packet
3. Actual data (encrypted)
4. Block encryption algorithms (like AES-CBC) require data to be in fixed chunks (e.g., 16 bytes). If your payload is only 3 bytes, we add random garbage data to fill the block. It also adds a security bonus as: If every time you typed a password the packet was exactly the same size, an attacker might guess it. Random padding varies the size slightly.
5. This is the digital wax seal. It is a checksum calculated after encryption (usually). If a hacker flips a single bit in the packet during transit, the MAC check will fail, and SSH will instantly disconnect to protect us.

The Cryptographic Handshake

We have covered the fundamental parts of the Transport Layer and what it does, here we will understand how secure connection is actually established between the user and the server and understand why Transport Layer is called one of the most important layers of SSH. This layer covers 4 essential aspects for a secure connection to be initiated:

1. The first thing the both sides do is exchange a plain text banner, which means they are exchanging their protocol versioning to each other. This message cannot be encrypted because encryption key do not exist at this point.
2. In this step SSH key exchange (also known as KEX) is used by the client and server to exchange supported algorithms to each other. The list is sent ordered by preference and the first matching algorithm is selected.

3. Once both the client and server have agreed on the same key exchange algorithm during algorithm negotiation, the key exchange phase can begin. Modern SSH implementations typically use ephemeral Elliptic Curve Diffie–Hellman (ECDH).
   The client begins by generating an , consisting of a private key and its associated public key. This key pair exists only for the duration of the handshake and is destroyed immediately after use. The client then sends its ephemeral public key to the server using the SSH_MSG_KEX_ECDH_INIT message.
   Upon receiving this message, the server generates its own ephemeral key pair and computes a shared secret using the client's public key and its own private key. Independently, the client computes the same shared secret using the server's public key and its private key. Due to the mathematical properties of Diffie–Hellman, both sides arrive at an identical shared secret value without ever transmitting that secret over the network.
   The resulting value, conventionally referred to as , is not an encryption key itself. Instead, it serves as raw keying material that will later be combined with other handshake data to derive multiple symmetric encryption and integrity keys. Because ephemeral keys are used, the compromise of long term keys does not allow past sessions to be decrypted, providing forward secrecy.
   At this stage, confidentiality against passive attackers is achieved, but the server has not yet been authenticated. Active man in the middle attacks are still possible until the shared secret is cryptographically bound to the server's host key in the subsequent step.
4. Now that the both sides have , the server must prove *it is who it says it is.* Here the concepts of host key comes into play.
   Both client and server independently calculate a hash (usually SHA-256) including information such as : client's public key, server's public key, selected algorithm, shared secret, protocol versions. This "" is a unique blueprint of this handshake.
   The server takes this hash H and signs it using it's private key. It sends this signature along with the public host key, back to the client in a `SSH_MSG_KEX_ECDH_REPLY` message.
   After the client receives the message, it looks at the public host key. It checks it's known_hosts file to see if it recognizes the sender, using the same public key the client checks if the signature is valid and that the sender is actually the server.
   This verification ensures that the connection is secure from any Man-In-Middle attacks.
5. After all this, this is the last step before bulk data encryption begins. Up until this point we have shared secret Key "K" and hash "H", using them both sides use a Key Derivation Function (KDF) to generate six distinct keys:
   Initialization Vector for encryption.

   The actual AES/ChaCha20 key.

   For the MAC (Message Authentication Code).

The reason why we create these keys on both directions (server & client) is for best security purpose. Imagine one of your key is compromised then the attacker will only be able to access single point of traffic.

To wrap up the Transport Layer, both sides send a tiny but critical message: `SSH_MSG_NEWKEYS`.

This message tells the other side: “Everything I send after this message will be encrypted using the keys we just derived.” Once both sides have sent and received this, the **Secure Tunnel** is officially open.

The handshake is now complete and we have understood about what Transport Layer is all about.

Conclusion :

At this point in the protocol, the SSH Transport Layer has completed its job.

Both client and server have agreed on algorithms, securely derived shared secrets, authenticated the server’s identity, and switched to encrypted communication using freshly generated symmetric keys. From this moment onward, every byte exchanged is protected for confidentiality and integrity, even over an untrusted network.

What the Transport Layer deliberately does **not** solve is identity on the client side. The server knows it is talking to *someone* over a secure tunnel, but it has no idea *who* that someone is. Encryption alone does not imply trust or authorization.

That responsibility belongs to the next stage of SSH.

In Part 2, we will move inside this encrypted tunnel to examine how SSH authenticates users, how public keys and signatures are validated, and how a single secure connection is multiplexed into channels for shells, port forwarding, and file transfers.

References
https://goteleport.com/blog/ssh-handshake-explained/
https://datatracker.ietf.org/doc/html/rfc4253